

LEVEL 2 — Pandas Series

Easy Line-by-Line Code, Output and Interview Guide

1. What is Series?

A Series is a one-dimensional labeled data structure. Think of it as one column of data with an index.

Code:

```
import pandas as pd

s = pd.Series([10, 20, 30])
print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Interview Tip: Series = values + index. Default index starts at 0.

2. Creating Series

Use `pd.Series()` to create a Series.

Code:

```
s = pd.Series([10, 20, 30])
print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Interview Tip: `pd.Series(data)` converts data into a Series.

3. Series from List

A Python list can be converted directly.

Code:

```
marks = [80, 90, 75]
s = pd.Series(marks)
print(s)
```

Output:

```
0    80
1    90
2    75
dtype: int64
```

Interview Tip: List values become Series values.

4. Series from NumPy Array

A NumPy array can be converted into a Series.

Code:

```
import numpy as np

arr = np.array([10, 20, 30])
s = pd.Series(arr)
print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Interview Tip: Pandas works closely with NumPy.

5. Series from Dictionary

Dictionary keys become the index and values become Series values.

Code:

```
data = {'Math': 90, 'English': 80, 'Science': 85}
s = pd.Series(data)
print(s)
```

Output:

```
Math      90
English   80
Science   85
dtype: int64
```

Interview Tip: Remember: key = index, value = data.

6. Custom Index

You can create your own index labels.

Code:

```
s = pd.Series([90, 80, 85],
              index=['Math', 'English', 'Science'])
print(s)
```

Output:

```
Math      90
English   80
Science   85
dtype: int64
```

Interview Tip: Custom labels make data easier to understand.

7. index

index shows the labels of the Series.

Code:

```
s = pd.Series([10, 20, 30])
print(s.index)
```

Output:

```
RangeIndex(start=0, stop=3, step=1)
```

Interview Tip: index tells you how rows are labeled.

8. values

values returns the actual data values.

Code:

```
s = pd.Series([10, 20, 30])
print(s.values)
```

Output:

```
[10 20 30]
```

Interview Tip: values gives the data without the index.

9. dtype

dtype tells you the data type of the values.

Code:

```
s = pd.Series([10, 20, 30])
print(s.dtype)
```

Output:

```
int64
```

Interview Tip: dtype helps you understand what kind of data is stored.

10. name

name gives a name to the Series.

Code:

```
s = pd.Series([90, 80, 85], name='Marks')
print(s.name)
```

Output:

```
Marks
```

Interview Tip: A Series name is useful when it represents a column.

11. size

size gives the total number of elements.

Code:

```
s = pd.Series([10, 20, 30, 40])
print(s.size)
```

Output:

```
4
```

Interview Tip: size counts all elements.

12. shape

shape gives the dimensions of the Series.

Code:

```
s = pd.Series([10, 20, 30, 40])
print(s.shape)
```

Output:

```
(4,)
```

Interview Tip: (4,) means one dimension containing 4 elements.

13. ndim

ndim gives the number of dimensions.

Code:

```
s = pd.Series([10, 20, 30])
print(s.ndim)
```

Output:

```
1
```

Interview Tip: A Series is one-dimensional.

14. Accessing Values

Use iloc for position-based access.

Code:

```
s = pd.Series([100, 200, 300])
print(s.iloc[0])
print(s.iloc[1])
```

Output:

```
100
200
```

Interview Tip: iloc uses position.

15. Positive Indexing

Positive positions start at 0.

Code:

```
s = pd.Series([10, 20, 30, 40])
print(s.iloc[0])
print(s.iloc[2])
```

Output:

```
10
20
```

Interview Tip: Position 0 = first; position 2 = third.

16. Negative Indexing

Negative positions count from the end.

Code:

```
s = pd.Series([10, 20, 30, 40])
print(s.iloc[-1])
print(s.iloc[-2])
```

Output:

```
40
30
```

Interview Tip: -1 = last; -2 = second last.

17. Label Indexing

Use custom labels to access values.

Code:

```
s = pd.Series([90, 80, 85],
              index=['Math', 'English', 'Science'])
print(s['Math'])
print(s['Science'])
```

Output:

```
90
85
```

Interview Tip: Label indexing uses the index name.

18. Slicing

iloc slicing uses positions; the stop position is excluded.

Code:

```
s = pd.Series([10, 20, 30, 40, 50])
print(s.iloc[1:4])
```

Output:

```
1    20
2    30
3    40
dtype: int64
```

Interview Tip: iloc[1:4] selects positions 1, 2 and 3.

19. Boolean Filtering

A condition selects only matching values.

Code:

```
s = pd.Series([40, 60, 80, 30])
result = s[s > 50]
print(result)
```

Output:

```
1    60
2    80
dtype: int64
```

Interview Tip: First create True/False conditions, then keep True values.

20. head()

head() shows the first 5 rows by default. You can specify a number.

Code:

```
s = pd.Series([10, 20, 30, 40, 50, 60])
```

```
print(s.head(3))
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

Interview Tip: head(3) = first 3 rows.

21. tail()

tail() shows the last 5 rows by default.

Code:

```
s = pd.Series([10, 20, 30, 40, 50, 60])
print(s.tail(3))
```

Output:

```
3    40
4    50
5    60
dtype: int64
```

Interview Tip: tail(3) = last 3 rows.

22. describe()

describe() gives a statistical summary for numeric data.

Code:

```
s = pd.Series([10, 20, 30, 40, 50])
print(s.describe())
```

Output:

```
count    5.000000
mean     30.000000
std      15.811388
min      10.000000
25%      20.000000
50%      30.000000
75%      40.000000
max      50.000000
```

Interview Tip: count = number of values; mean = average; std = standard deviation; min/max = smallest/largest; 25%, 50%, 75% = quartiles.

Quick Interview Revision

Q: What is a Series?

A: A one-dimensional labeled data structure.

Q: Series vs DataFrame?

A: Series is one-dimensional; DataFrame is two-dimensional.

Q: How do you create a Series?

A: `pd.Series(data)`

Q: Dictionary Series?

A: Keys become indexes and values become data.

Q: What is `iloc`?

A: Position-based indexing.

Q: What is label indexing?

A: Accessing data using an index label.

Q: What is boolean filtering?

A: Selecting values using a True/False condition.

Q: What does `head()` do?

A: Shows the first 5 rows by default.

Q: What does `tail()` do?

A: Shows the last 5 rows by default.

Q: What does `describe()` do?

A: Provides descriptive statistics for numeric data.

Final Practice Program

```
import pandas as pd

marks = pd.Series(
    [85, 90, 75, 95],
    index=['Math', 'English', 'Science', 'Computer'],
    name='Marks'
)

print(marks)
print("Index:", marks.index)
print("Values:", marks.values)
print("Dtype:", marks.dtype)
print("Size:", marks.size)
print("Shape:", marks.shape)
print("Dimensions:", marks.ndim)
print("Math:", marks['Math'])
print("High marks:")
print(marks[marks > 80])
print("First 2:")
print(marks.head(2))
print("Last 2:")
print(marks.tail(2))
print("Summary:")
print(marks.describe())
```

Memory trick: Series = one-dimensional labeled data. Properties: index, values, dtype, name, size, shape, ndim. Selection: `iloc` = position, label = index name, boolean filtering = condition.